

SHOP MANAGEMENT APP — MASTER TECHNICAL ARCHITECTURE

Final Master Baseline

Stack: Flutter + GetX + Supabase (Postgres + Auth + Storage) + GetStorage **Scope:** Single-shop (single tenant), Android + iOS (Windows optional later)

This is the single implementation baseline for the project. It defines the application architecture, business rules, data model, security model, user flows, Flutter structure, and implementation order — so you can build against it without re-deciding things mid-development.

1. Executive Summary

The Shop Management App is a single-shop retail management system built with Flutter, GetX, and Supabase. It handles inventory, purchases, sales, customers (khata/credit), suppliers, expenses, and reports for one shop and its staff.

- One shop, one Supabase project — no multi-tenant isolation logic needed.
 - Shop Owner/Admin has full access; Staff has restricted operational access (configurable).
 - Supabase Auth manages credentials; passwords are never stored in your own tables.
 - Postgres is the primary database; Row Level Security (RLS) is the authorization boundary.
 - Business-critical operations (sale, purchase, khata payment) run as **Postgres functions (RPC)** called atomically — never as separate client-side writes.
 - GetX manages state, routing, and dependency injection. GetStorage handles local cache/session/preferences only — never the source of truth.
-

2. Architecture Decisions

Area	Final Decision
Frontend	Flutter single codebase (Android + iOS)
State Management	GetX
Navigation	GetX routing, centralized route names
Authentication	Supabase Auth (email/password)
Database	Supabase Postgres
File Storage	Supabase Storage (product images, shop logo)
Serverless Backend	Supabase Postgres Functions (RPC) for atomic transactions — no separate Edge Functions required unless a future need (e.g. scheduled reports) arises
Multi-Tenancy	Not required — single shop, single Supabase project
Local Cache	GetStorage (session flags, preferences, offline product cache)
Security	Supabase Auth + Row Level Security (RLS) policies
UI Architecture	Feature-first Flutter structure: bindings, controllers, models, repositories, services, views, widgets per feature

3. User Roles

Role	Scope	Responsibilities
Owner/Admin	Whole shop	Manage inventory, categories, purchases, sales, khata, customers, suppliers, expenses, reports, staff, settings
Staff	Assigned permissions only	Perform permitted operational actions (typically: create sales, view products/stock); cannot manage staff, cannot edit prices/costs unless explicitly permitted

3.1 Role Separation

- Role checks in Flutter (hiding buttons/screens) are for UX only.
- **Supabase RLS policies must enforce the real authorization** — a staff account must be blocked at the database level from actions it shouldn't perform, even if the UI is bypassed (e.g. via API call).

- Staff cannot alter stock directly outside the controlled sale/purchase RPC functions.
 - Staff cannot manage other staff accounts or shop settings.
-

4. App Lifecycle (Single Shop)

1. Owner installs app, signs up (Supabase Auth creates the identity).
2. On first signup, app creates the `shop_profile` row (one row — this is a single-shop app) and a `users` row with `role = 'owner'`, `is_active = true`.
3. Owner logs in → session guard checks `is_active` → routes to Dashboard.
4. Owner adds staff members from **Staff Management** screen: creates a Supabase Auth user (via invite/email) + a `users` row with `role = 'staff'`, `is_active = true`, and selected permissions.
5. Staff logs in → session guard checks `is_active` and role → routes to Dashboard (with restricted menu based on role/permissions).
6. If Owner deactivates a staff member (`is_active = false`), that staff is blocked from login/operational access immediately (checked at session guard + enforced by RLS).

No "pending shop activation" or "Platform Admin approval" step is needed here — this is intentionally simpler than a multi-tenant SaaS since there's only one shop.

5. Supabase Database Schema

All tables live in the `public` schema of a single Supabase project. No `shop_id` column is required since there is only one shop — but keep an eye on section 24 (future-proofing) if you might go multi-shop later.

5.1 shop_profile (single row)

Field	Type	Required	Purpose
id	uuid (PK)	Yes	Fixed single row id
name	text	Yes	Shop name
owner_name	text	Yes	Owner display name
phone	text	No	Shop phone
address	text	No	Shop address
currency_code	text	Yes	Default PKR
currency_symbol	text	Yes	Default "Rs."
logo_url	text	No	Supabase Storage URL
created_at	timestampz	Yes	Creation time
updated_at	timestampz	Yes	Last update

5.2 **users** (staff/owner membership)

Field	Type	Required	Purpose
id	uuid (PK, = auth.uid())	Yes	Matches Supabase Auth user id
name	text	Yes	Display name
email	text	Yes	Auth email
role	text	Yes	owner / staff
permissions	jsonb	No	Fine-grained permission flags for staff (e.g. <code>{"can_edit_price": false}</code>)
is_active	bool	Yes	Access flag
phone	text	No	Optional
created_at	timestamptz	Yes	Creation
updated_at	timestamptz	Yes	Last update
created_by	uuid	No	Creator user id

5.3 **categories**

Field	Type	Purpose
id	uuid (PK)	
name	text	Category name
is_active	bool	Soft-delete/visibility
created_at / updated_at	timestamptz	
created_by	uuid	

5.4 **products**

Field	Type	Purpose
id	uuid (PK)	
article	text	Product/article name
category_id	uuid (FK)	Category reference
color	text	Optional variant
size	text	Optional variant
barcode	text (unique)	Scan/search
unit	text	piece/kg/box/etc.
purchase_price	numeric	Current/default cost
sale_price	numeric	Current/default sale rate
current_stock	numeric	Current quantity
min_stock_threshold	numeric	Low-stock alert threshold
image_url	text	Supabase Storage URL
is_active	bool	Soft-delete flag
created_at / updated_at	timestampz	
created_by / updated_by	uuid	

5.5 **purchases** + **purchase_items**

purchases

Field	Purpose
id (PK)	
purchase_no	Human-readable number
supplier_id (FK)	Optional supplier
supplier_name	History snapshot
total_amount	Sum of items
paid_amount	Amount paid at purchase time
due_amount	Remaining payable
notes	Optional
created_by	Creator
created_at	Timestamp

purchase_items

Field	Purpose
id (PK)	
purchase_id (FK)	
product_id (FK)	
product_name	History snapshot
quantity	Purchased qty
rate	Purchase rate
subtotal	$\text{quantity} \times \text{rate}$

5.6 **sales** + **sale_items**

sales

Field	Purpose
id (PK)	
sale_no	Human-readable number
customer_id (FK)	Required for credit sales
customer_name	History snapshot
total_amount	Sum of items
paid_amount	Amount paid
due_amount	Remaining receivable
payment_type	☐ cash / ☐ credit
staff_id (FK)	Creator
staff_name	History snapshot
created_at	Timestamp

sale_items

Field	Purpose
id (PK)	
sale_id (FK)	
product_id (FK)	
product_name	History snapshot
quantity	Sold qty
rate	Actual sale rate
purchase_rate_at_sale	Cost snapshot — for historical profit accuracy
subtotal	$\text{quantity} \times \text{rate}$
profit	$(\text{rate} - \text{purchase_rate_at_sale}) \times \text{quantity}$

5.7 **customers** (Khata)

Field	Purpose
id (PK)	
name	
phone	
total_balance	Outstanding credit (receivable)
is_active	
last_transaction_at	
created_at / updated_at	

5.8 **khata_payments**

Field	Purpose
id (PK)	
customer_id (FK)	
customer_name	Snapshot
amount	Amount received
note	Optional
received_by (FK)	Staff/owner who received it
received_by_name	Snapshot
created_at	Timestamp

5.9 **suppliers**

Field	Purpose
id (PK)	
name	
phone	
address	
balance	Outstanding payable to supplier
created_at / updated_at	

5.10 **expenses**

Field	Purpose
id (PK)	
category	e.g. Rent, Utility, Salary
description	
amount	
created_by (FK)	
created_at	

5.11 `activity_logs`

Field	Purpose
id (PK)	
user_id (FK)	Actor
user_name	Snapshot
role	Actor role
action	e.g. <code>sale_created</code> , <code>purchase_created</code> , <code>staff_deactivated</code>
entity_type	sale/product/customer/etc.
entity_id	Affected row id
description	Human-readable audit text
metadata	jsonb, small structured data
created_at	Server timestamp

6. Critical Business Transactions (Postgres RPC Functions)

Stock, sales, purchases, and khata balances are high-risk data — a half-applied write (e.g. sale row created but stock not decreased) corrupts your numbers. **Every one of these must be a single Postgres function called via `supabase.rpc(...)`, wrapped in a transaction, never split into multiple client-side `.insert()`/`.update()` calls.**

6.1 Purchase Transaction (`create_purchase` RPC)

1. Validate product exists and is active.
2. Validate quantity > 0.
3. Calculate `total = quantity × rate`.
4. Insert `purchases` row.
5. Insert `purchase_items` row(s).
6. Increase `products.current_stock` by quantity.
7. Optionally update `products.purchase_price` to latest rate (business rule — confirm with client).
8. If `due_amount > 0` and supplier set, increase `suppliers.balance`.
9. Insert `activity_logs` row.
10. Commit atomically — all steps succeed or none do.

6.2 Sale Transaction (`create_sale` RPC)

1. Validate product exists, is active, and `current_stock >= requested quantity`.
2. Calculate `total = quantity × sale rate`.
3. Capture `purchase_rate_at_sale` from the product's current `purchase_price`.
4. Calculate `profit = (sale rate - purchase_rate_at_sale) × quantity`.
5. Insert `sales` row + `sale_items` row(s).
6. Decrease `products.current_stock` by quantity.
7. If `payment_type = 'credit'`, increase `customers.total_balance` by `sale.total`.
8. Insert `activity_logs` row.
9. Commit atomically.

6.3 Khata Payment Transaction (`create_khata_payment` RPC)

1. Validate customer exists and is active.
2. Validate `amount > 0`.
3. Optionally validate `amount <= outstanding balance` (unless overpayment is allowed).
4. Insert `khata_payments` row.
5. Decrease `customers.total_balance`.
6. Update `customers.last_transaction_at`.
7. Insert `activity_logs` row.
8. Commit atomically.

Implementation note: write these as `plpgsql` functions with `SECURITY DEFINER` (carefully scoped) or call them through RLS-compliant policies so a staff account can execute the RPC without needing direct table UPDATE grants on `products`/`customers`.

7. Inventory & Profit Rules

Rule	Implementation
Stock after purchase	$\text{current_stock} + \text{quantity}$
Stock after sale	$\text{current_stock} - \text{quantity}$
Low stock	$\text{current_stock} \leq \text{min_stock_threshold}$
Sale total	$\text{quantity} \times \text{actual sale rate}$
Profit per unit	$\text{sale rate} - \text{purchase_rate_at_sale}$
Sale profit	$(\text{sale rate} - \text{purchase_rate_at_sale}) \times \text{quantity}$
Customer balance after credit sale	$\text{total_balance} + \text{sale.total}$
Customer balance after payment	$\text{total_balance} - \text{payment.amount}$
Historical profit	Always use $\text{purchase_rate_at_sale}$ stored on the sale item — never recompute from current product cost

Use `numeric` (not `float`) columns in Postgres for all money/price fields to avoid rounding errors.

8. Row Level Security (RLS) Strategy

Supabase RLS policies are your real authorization boundary — Flutter role checks are UX only.

- Enable RLS on **every** table.
- `shop_profile`: readable by any authenticated user of this app; writable only by `role = 'owner'`.
- `users`: a user can read their own row; only `owner` can insert/update/deactivate other users.
- `products`, `categories`: readable by any active authenticated user; write access (price/cost edits) restricted to `owner` unless a staff permission flag explicitly allows it.
- `sales`, `sale_items`: insert allowed only through the `create_sale` RPC (not direct insert) for `staff`; `owner` can also view/edit history if needed.
- `purchases`, `purchase_items`, `khata_payments`: same RPC-only pattern; typically `owner`-only, unless staff is explicitly permitted to record purchases.

- `activity_logs`: insert-only for authenticated users (via RPC), no update/delete from the client at all.
- Every policy should check `auth.uid()` against the `users` table to confirm `is_active = true` before allowing any read/write.
- Deactivated (`is_active = false`) users must be blocked by RLS even if their session token is still technically valid until it expires.

8.1 Access Condition

Operational access (client-side + enforced by RLS) requires:

```
auth.uid() exists AND users.is_active = true
```

This must be checked both at the GetX session guard (for good UX/routing) **and** inside RLS policies (for real security).

9. Flutter Project Structure (Feature-First)

```
lib/
├─ main.dart
├─ app/
│   ├─ app.dart
│   ├─ routes/
│   │   ├─ app_routes.dart
│   │   └─ app_pages.dart
│   ├─ bindings/
│   │   └─ initial_binding.dart
│   └─ theme/
│       ├─ app_theme.dart
│       ├─ app_colors.dart
│       └─ app_text_styles.dart
│
├─ core/
│   ├─ config/
│   │   └─ env_config.dart           # Supabase URL/anon key, per-env if needed
│   ├─ constants/
│   ├─ enums/
│   ├─ errors/
│   ├─ extensions/
│   ├─ utils/
│   │   ├─ validators.dart
│   │   └─ formatters.dart
│   └─ services/
│       ├─ supabase_service.dart     # Supabase client singleton
│       ├─ storage_service.dart      # GetStorage wrapper
│       └─ pdf_service.dart          # Invoice generation
│
├─ features/
│   └─ auth/
│       ├─ bindings/
│       ├─ controllers/
│       ├─ models/
│       ├─ repositories/
│       ├─ services/
│       └─ views/
│
└─ widgets/
    ├─ staff/
    ├─ dashboard/
    ├─ categories/
    ├─ products/
    ├─ purchases/
    ├─ sales/
    ├─ khata/                        # customers + payments
    ├─ suppliers/
    ├─ expenses/
    └─ reports/
```

```

|   |─ activity_log/
|   |─ shop_profile/
|
|   └─ shared/
|       |─ models/
|       |─ widgets/
|       |   |─ custom_appbar.dart
|       |   |─ custom_button.dart
|       |   |─ custom_textfield.dart
|       |   |─ custom_dropdown.dart
|       |   |─ summary_card.dart
|       |   |─ low_stock_badge.dart
|       |   |─ empty_state.dart
|       |   |─ confirm_dialog.dart
|       |   └─ date_range_picker.dart
|       └─ components/

```

9.1 Feature Pattern

Every feature folder follows the same internal structure:

- `bindings/` — dependency registration and controller lifecycle
- `controllers/` — UI state and user actions (`GetxController`, `.obs` state)
- `models/` — feature data models (`fromJson`/`toJson`) matching Supabase tables)
- `repositories/` — data contracts (interfaces controllers depend on)
- `services/` — actual Supabase calls (RPC calls, `.select()`, `.insert()`, storage upload)
- `views/` — pages/screens (pure UI, no direct Supabase calls)
- `widgets/` — reusable feature-specific UI

Rule: Views never call Supabase directly. Controllers never call Supabase directly. Only `services/` touch `supabase_service.dart`.

10. Authentication & Session Guard

10.1 Responsibilities

- Supabase Auth creates and authenticates identities (email/password).
- Your `users` table stores role, permissions, and `is_active` — the actual application-level state.
- Passwords are never stored in your own tables.
- After sign-in, the app loads the current user's `users` row.
- Routing decisions are based on that combined state.

10.2 Routing Flow

```
Splash → Supabase.initialize() → auth state stream
  → Signed out → Login screen
  → Signed in, no users row → Recovery/Contact Owner screen
  → Signed in, is_active = false → Access Denied screen
  → Signed in, is_active = true → Dashboard (menu scoped by role/permissions)
```

11. GetX Implementation Rules

- Use `GetMaterialApp`.
- Centralize route names in `app_routes.dart`; map routes + bindings in `app_pages.dart`.
- Use `Bindings` per feature — `Get.lazyPut()` for controllers/services.
- Keep Supabase calls out of widgets — always through `services/`.
- Use `Obx`/`GetBuilder` intentionally on specific widgets, not the whole screen.
- Use `Get.offAllNamed()` after login/logout so auth screens leave the navigation stack.
- Keep business calculations (profit, totals, balances) in controllers/services, never in widget build methods.
- Dispose controllers through normal GetX lifecycle (`onClose()`), don't hold heavy listeners manually.

12. UI & Navigation Requirements

12.1 Auth Screens

- Splash
- Login
- Forgot Password

12.2 Operational Screens

- Dashboard
- Categories
- Products (list, add/edit, barcode scan/search)
- Purchases (entry, history)
- Sales / POS (cart entry, history)
- Khata (customers, ledger, record payment)
- Suppliers (list, ledger)
- Expenses
- Reports (Daily/Monthly/P&L/Stock/Best-sellers/Outstanding credit)
- Invoice preview/print/share

- Staff Management (Owner only)
- Activity Log (Owner only)
- Shop Profile / Settings

13. Dashboard Requirements

- Today's sales total
- Low-stock alerts (list of products at/below threshold)
- Outstanding credit total (khata)
- Quick Sale shortcut
- Quick Purchase shortcut (Owner)
- Recent activity feed
- Recent sales list
- Sales trend chart (this week/month)

Avoid one dashboard query that reads the entire `sales` table. Use date-filtered queries (`created_at >= start_of_day`) and Postgres views for aggregates if data volume grows.

14. Reports

Report	Query / Calculation
Daily Sales	<code>sales</code> where <code>created_at</code> within today; <code>SUM(total_amount)</code>
Weekly/Monthly Sales	Same, filtered by week/month range
Profit	<code>SUM(sale_items.profit)</code> within period
Best Sellers	Group <code>sale_items</code> by <code>product_id</code> , <code>SUM(quantity)</code> desc
Outstanding Credit	<code>customers</code> where <code>total_balance > 0</code> ; <code>SUM(total_balance)</code>
Stock Report	<code>products</code> where <code>is_active = true</code> , flag rows where <code>current_stock <= min_stock_threshold</code>
Supplier Payable	<code>SUM(suppliers.balance)</code>

15. GetStorage (Local Storage) — What to Store

Key	Purpose
is_logged_in	Skip login if session valid
user_id, user_role, permissions	Quick access without re-fetching on every screen
cached_products	Offline product list for fast POS search
theme_mode	Dark/light preference
last_sync_time	For offline-sync/cache-refresh logic
low_stock_threshold_default	User-configurable default

Rule: GetStorage = cache & preferences only. Source of truth is always Supabase. Never store financial transaction data only locally.

16. Permissions Required

Permission	Why
Camera	Barcode scanning
Storage/Photos	Product image upload, saving invoice PDF
Internet	Supabase sync
Notifications	Low-stock alerts (Android 13+ needs runtime request)

17. Reusable Widgets

Widget	Used In
CustomAppBar	All screens
CustomButton (with loading state)	Forms, POS
CustomTextField (with validator)	All forms
CustomDropdown	Category/customer/supplier select
SummaryCard	Dashboard stats
ProductCard	Product grid/list
LowStockBadge	Product list + Dashboard
EmptyState	Any empty list
LoadingIndicator	Global loading overlay
ConfirmDialog	Delete/edit confirmations
DateRangePicker	Reports filtering
BarcodeScannerView	Product search/add
InvoiceTemplate	Sale invoice PDF/preview

18. Indexes & Query Planning (Postgres)

- sales(created_at)
- sales(customer_id, created_at)
- sales(staff_id, created_at)
- sale_items(product_id)
- purchases(created_at)
- activity_logs(created_at)
- customers(total_balance)
- products(category_id, is_active)
- products(is_active, current_stock) — for low-stock filtering
- products(barcode) — unique index for fast scan lookup

Add indexes only for query patterns you actually use — check with `EXPLAIN ANALYZE` if a report screen feels slow.

19. Soft Delete & History

- Deactivate products (`(is_active = false)`) rather than deleting ones with transaction history.
 - Deactivate customers/suppliers with balances/history rather than deleting.
 - Deactivate staff (`(is_active = false)`) instead of deleting — preserves audit trail.
 - Never casually delete `(sales)`/`(purchases)` rows.
 - Reverse a sale using a controlled reversal/adjustment RPC (re-adds stock, reverses balance), not a manual delete.
-

20. Error Handling & Reliability

Scenario	Expected Behavior
Insufficient stock	Block sale, show current available stock
Network unavailable	Show clear error; never fake success
RPC/transaction conflict	Retry safely or ask user to retry
Invalid quantity	Block before write (client-side validation + DB check)
Credit sale without customer	Block before write
Duplicate submit	Disable button during write; consider idempotency key for RPC
Unauthorized write	RLS rejects it; show a permission error, not a generic crash
Deactivated staff login attempt	Block at session guard + RLS

The UI must clearly distinguish "saved successfully" from "save failed." Never show an invoice/success screen until the RPC call has actually returned success.

21. Repository & Service Responsibilities

Repository	Responsibilities
AuthRepository	Login, logout, current session, password reset
ShopProfileRepository	Shop profile read/update
UserRepository (Staff)	Staff CRUD, activate/deactivate, permissions
ProductRepository	Product/category CRUD, stock queries, barcode search
PurchaseRepository	Calls create_purchase RPC, purchase history queries
SaleRepository	Calls create_sale RPC, sale history queries
CustomerRepository	Customer CRUD, balance reads
KhataRepository	Calls create_khata_payment RPC
SupplierRepository	Supplier CRUD, balance reads
ExpenseRepository	Expense CRUD
ReportRepository	Report queries/aggregates
ActivityLogRepository	Audit entries and history

UI calls controllers → controllers call repositories → repositories call services (which touch Supabase). Never skip a layer.

22. Dev / Prod Environment

Environment	Supabase Project	Purpose
DEV	shop-management-dev	Development and test data
PROD	shop-management-prod	Real shop data

- Use separate Supabase projects for dev and prod — never test against production data.
- Store URL/anon key via `--dart-define` or a `.env` (not committed) loaded through `env_config.dart`.
- Example: `flutter run --dart-define=SUPABASE_URL=... --dart-define=SUPABASE_ANON_KEY=...`

- Protect the production **service role key** — it must never ship inside the Flutter app; only the anon key (governed by RLS) goes client-side.
-

23. Security Model

- Supabase Auth is the identity layer.
 - RLS policies are the primary database authorization boundary — not Flutter code.
 - `is_active` must be checked both client-side (routing/UX) and inside RLS (real enforcement).
 - Staff cannot alter stock directly outside the sale/purchase RPCs.
 - Staff cannot manage other staff accounts.
 - Audit logs (`activity_logs`) preserve who performed sensitive actions — insert-only from the client.
 - Never commit the Supabase service role key to source control or ship it in the app bundle.
-

24. Implementation Phases

Phase	Area	Scope
1	Foundation	Flutter setup, GetX skeleton, theme, routing, Supabase project (dev)
2	Auth	Supabase Auth, login, password reset, session guard
3	Shop Profile & Staff	Shop profile setup, staff CRUD, roles/permissions
4	Security	RLS policies for every table, role restriction tests
5	Inventory	Categories, products, barcode, stock thresholds
6	Purchases	Purchase entry via <code>create_purchase</code> RPC, history
7	Sales	POS cart, <code>create_sale</code> RPC, invoice generation
8	Khata	Customers, <code>create_khata_payment</code> RPC, ledger screens
9	Suppliers & Expenses	CRUD + balance tracking
10	Dashboard	Summary cards, charts, low-stock alerts
11	Reports	Daily/monthly/P&L/stock/best-sellers/outstanding credit
12	Audit	Activity log screen and history
13	Hardening	Indexes, offline cache (GetStorage), error handling, release build

25. Testing Strategy

25.1 Auth & Access

- Login works only for `(is_active = true)` users.
- Deactivated staff is blocked from login/operational screens.
- Owner-only screens (Staff, Activity Log, Shop Profile) are inaccessible to Staff role.

25.2 Business Transactions

- Purchase increases stock exactly once, atomically.
- Sale decreases stock exactly once, atomically; blocked if insufficient stock.
- Credit sale increases customer balance atomically.
- Khata payment decreases customer balance atomically.
- Historical sale profit stays correct even after product's current purchase price later changes.
- Duplicate submit does not create duplicate rows.

25.3 Security (RLS)

- Staff cannot directly `(UPDATE)` `(products.current_stock)` or `(customers.total_balance)` outside the RPC functions.
- Staff cannot read/write `(users)` table rows other than their own.
- Deactivated users are blocked by RLS even with a still-valid token.

26. Final Architecture Flow

OWNER SIGNUP



SUPABASE AUTH USER CREATED



SHOP PROFILE ROW CREATED (single row)



OWNER USERS ROW CREATED (role = owner, is_active = true)



DASHBOARD



CATEGORIES • PRODUCTS • PURCHASES • SALES • KHATA • SUPPLIERS • EXPENSES • REPORTS •
STAFF



ACTIVITY LOG

27. Final Master Decision

This document is the single master technical baseline for implementation. The application is a Flutter + GetX + Supabase single-shop management system for Android + iOS.

There is no multi-tenant complexity and no platform-admin activation step — a shop's owner signs up and gets immediate access; staff accounts are created and managed by the owner directly. Operational access is controlled by `is_active` on the `users` row, enforced both at the GetX session guard (UX) and via Supabase RLS policies (real security). Critical stock, sales, purchases, and khata operations run through atomic Postgres RPC functions — never as separate client-side writes.

GetStorage is used only for local cache/session/preferences; Supabase Postgres remains the single source of truth at all times.